

Build a Data Warehouse on Microsoft Fabric

A toy shop, three layers, and ten steps.

What this is. A small project you can finish in a weekend. You take some shop till files, tidy them up, and turn them into a warehouse that a chart can read.

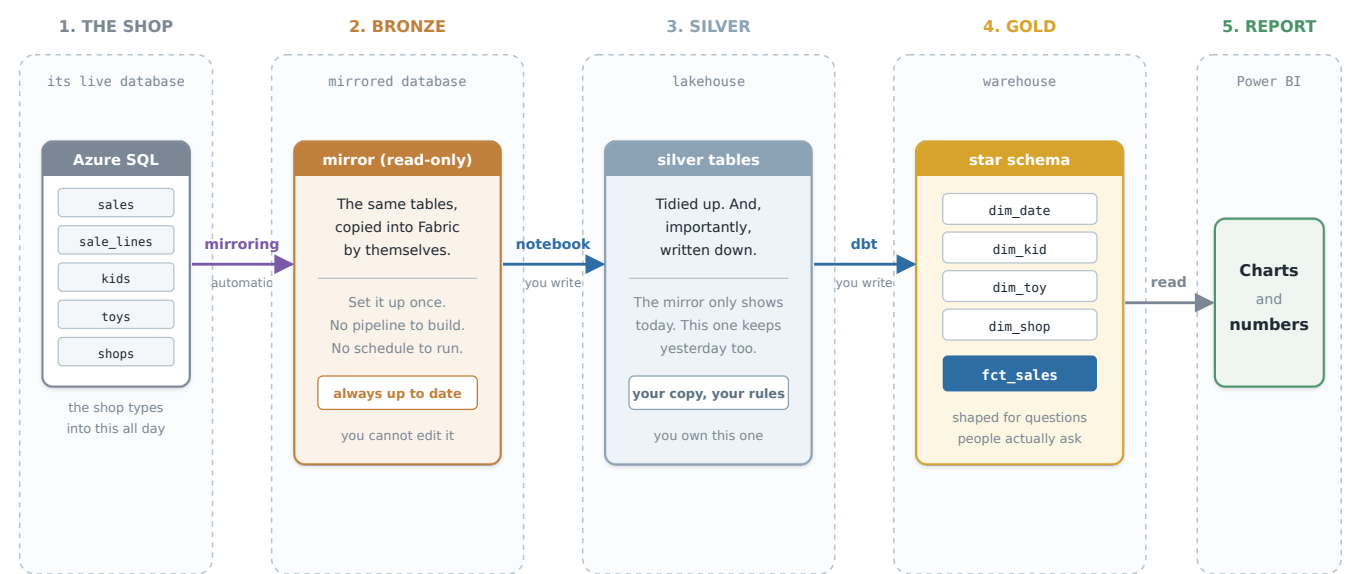
What you need first. Guide 05 and Guide 06. This is the same warehouse you built on your laptop, done for real on Fabric.

Written August 2026. Fabric's buttons and menus move around often. If a screen does not look like the one described here, check Microsoft Learn. The ideas do not move; only the buttons do.

1. What you are building

The shop runs a database. People type into it all day — sales, children, toys. You are not allowed to touch it, and you must not slow it down.

So Fabric makes its own copy, keeps it up to date by itself, and you work on the copy. The copy goes through three rooms on the way to a chart, and each room is only allowed to do one job.



Fabric gives you Bronze for free. You build Silver and Gold.
Bronze = what the shop has right now | Silver = what was true, and when | Gold = what people ask for

The whole project on one page. Left to right, start to finish.

Room	Its one job	Where it lives in Fabric	Who does the work
Bronze	A live copy of the shop's database	Mirrored database	Fabric, by itself
Silver	Tidy it, and write down what was true	Lakehouse	A Spark notebook you write
Gold	Answer questions	Warehouse	dbt

Why two different places?

A Lakehouse is good at cleaning up mess. A Warehouse is good at answering questions. So the mess gets cleaned in the Lakehouse, and the tidy result gets shaped in the Warehouse. Each job happens where the right tool already lives.

THE ONE RULE THAT SAVES YOU A WHOLE DAY

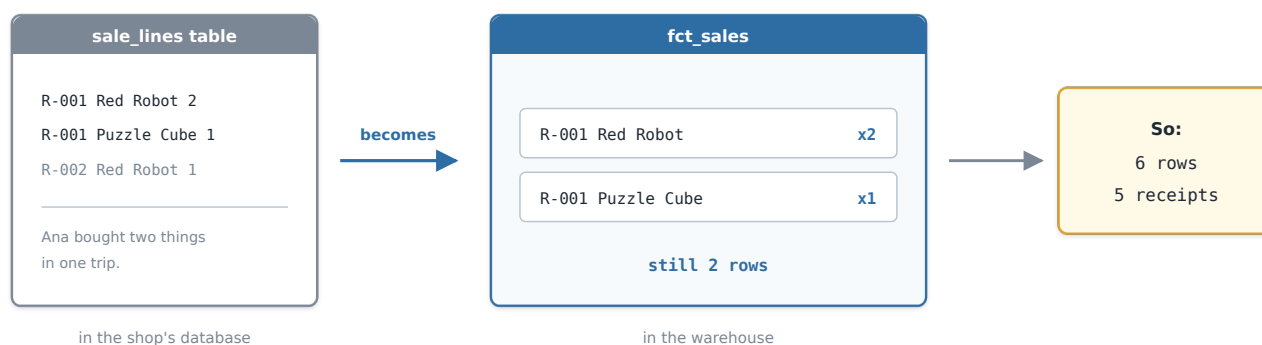
dbt cannot write to a Lakehouse. The Lakehouse can be read by SQL, but not written to by SQL. So dbt reads Silver and writes Gold — it never builds Silver.

If you point dbt at the Lakehouse, every model fails and the error sounds like a password problem. It is not. It is this.

2. The example

A toy shop. Three children, three toys, two shops, five receipts. Small enough to count on your fingers, which is the point — you can check every answer by hand.

One thing about this data matters more than the rest:



Ana bought two things in one trip. One receipt, two rows — in the shop's database and in yours.

So the shop had **5 sales** but the file has **6 rows**. Neither number is wrong. They answer different questions. Getting this straight now saves you from every counting mistake later.

SAY THIS OUT LOUD BEFORE YOU BUILD ANYTHING

"One row per toy per receipt." That sentence is your rule. Every table you build has to obey it. Write it down where you can see it.

3. Before you start

You need	Notes
A Fabric trial	Free for 60 days. That is plenty.
An Azure SQL database	Or SQL Server, PostgreSQL, Cosmos DB, or Snowflake. Mirroring works with all of them.
Every source table to have a primary key	Mirroring skips tables that do not have one. Check this first — it is the most common surprise.
To be workspace admin	Not a nice-to-have. dbt will not work without it.
Python on your laptop	For dbt. Version 3.9 or newer.

4. The ten steps

1 Make one workspace and three things inside it

In Fabric, make a new workspace. Inside it, make three items — one for each layer:

New	→	Mirrored database	→	call it	mirror_shop	(bronze)
New	→	Lakehouse	→	call it	lh_retail	(silver)
New	→	Warehouse	→	call it	wh_retail	(gold)

YOU SHOULD SEE

More items than you made. Each one comes with a hidden twin called a **SQL analytics endpoint** — same name, different icon. That twin is the door other things use to read it with SQL. You did not do anything wrong.

All three must be in the **same workspace**. If they are spread across workspaces, they cannot read each other and nothing later works.

2 Let the Warehouse see the Lakehouse

Open `wh_retail`. In the Explorer panel on the left, click **+ Warehouses** and pick the `lh_retail` SQL analytics endpoint.

Now run this in a new SQL query:

```
SELECT TOP 10 * FROM lh_retail.dbo.silver_sales;
```

YOU SHOULD SEE

An error saying **invalid object name**. That is the right error! It means the Warehouse can see the Lakehouse, and the table just is not built yet.

If instead it says **could not find database**, the two are not connected. Go back and add the endpoint again.

3 Switch mirroring on (Bronze)

Open `mirror_shop`. It asks for a connection to your database. Give it the server name and a login, choose which tables to copy, and press go.

Then wait. The first copy takes a few minutes. After that, changes arrive within about a minute, forever, without you doing anything.

YOU SHOULD SEE

A table of your tables, each one saying **Running**, with a row count next to it. Those counts should match your source database.

If a table says it was **skipped**, it almost certainly has no primary key. Mirroring cannot follow changes without one. Add a key at the source, or leave that table out.

Now go into `lh_retail` and make a **shortcut** to the mirrored tables. A shortcut is a pointer, not a copy — it lets your notebook read the mirror without moving anything.

```
lh_retail → Get data → New shortcut → Microsoft OneLake
          → pick mirror_shop → tick the tables you want
```

YOU SHOULD SEE

The shop's tables listed inside your Lakehouse, with a little link symbol on the icon. That symbol means "this lives somewhere else".

If Spark later refuses to read them, it is a permission called **Read all OneLake data** on the mirrored database. Being workspace admin is not always enough on its own.

WHAT THIS REPLACES

Everything you would normally build to get data in — a copy pipeline, a schedule, retry logic, someone on call — is now a page of settings. The background copying does not even use up your capacity.

It is worth pausing on that. Ingestion is usually the part that breaks most often, and here you did not write any.

4 Tidy it and write it down (Silver)

Make a notebook in the workspace and attach `lh_retail`. Silver has two jobs now. The first is the obvious one: tidy the data up. The second is the one mirroring creates: **keep a dated copy**, because the mirror forgets.

```
from pyspark.sql import functions as F

# 1. Read the mirror through the shortcut. It is already a proper table
#    with proper types - the shop's database did that work for us.
lines = spark.read.table("sale_lines")
toys = spark.read.table("toys")

# 2. Tidy. Much less to do than with files, but names still have spaces
#    and codes still come in mixed upper and lower case.
tidy = (lines
        .withColumn("receipt_no", F.trim("receipt_no"))
        .withColumn("toy_name", F.trim("toy_name"))
        .withColumn("sale_date", F.to_date("sold_at"))
        .withColumn("amount", F.col("quantity") * F.col("toy_price")))

# 3. Stamp it with the time we looked. This is the important bit.
tidy = tidy.withColumn("_seen_at", F.current_timestamp())

tidy.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable("silver")

print("rows:", tidy.count())
```

Sales rarely change once they happen, so overwriting is fine for them. Children and toys are different — those *do* change, and that is where the mirror forgetting actually hurts. For those, add today's version instead of replacing yesterday's:

```
# kids change: Ben moves house, and we want to remember both addresses
today = (spark.read.table("kids")
        .withColumn("_seen_at", F.current_timestamp()))

today.write.format("delta").mode("append").saveAsTable("silver_kids_history")
```

WHY THIS MATTERS MORE THAN IT LOOKS

With files, Bronze kept every version because the old files were still sitting there. With a mirror, there are no old files. The moment the shop updates Ben's address, the only record of where he used to live is the one **you** wrote down.

If you skip this, you can never answer "where did Ben live in July?" — and it will not fail loudly. It will just quietly be unanswerable.

YOU SHOULD SEE

rows: 6 and two new tables under `Tables/`.

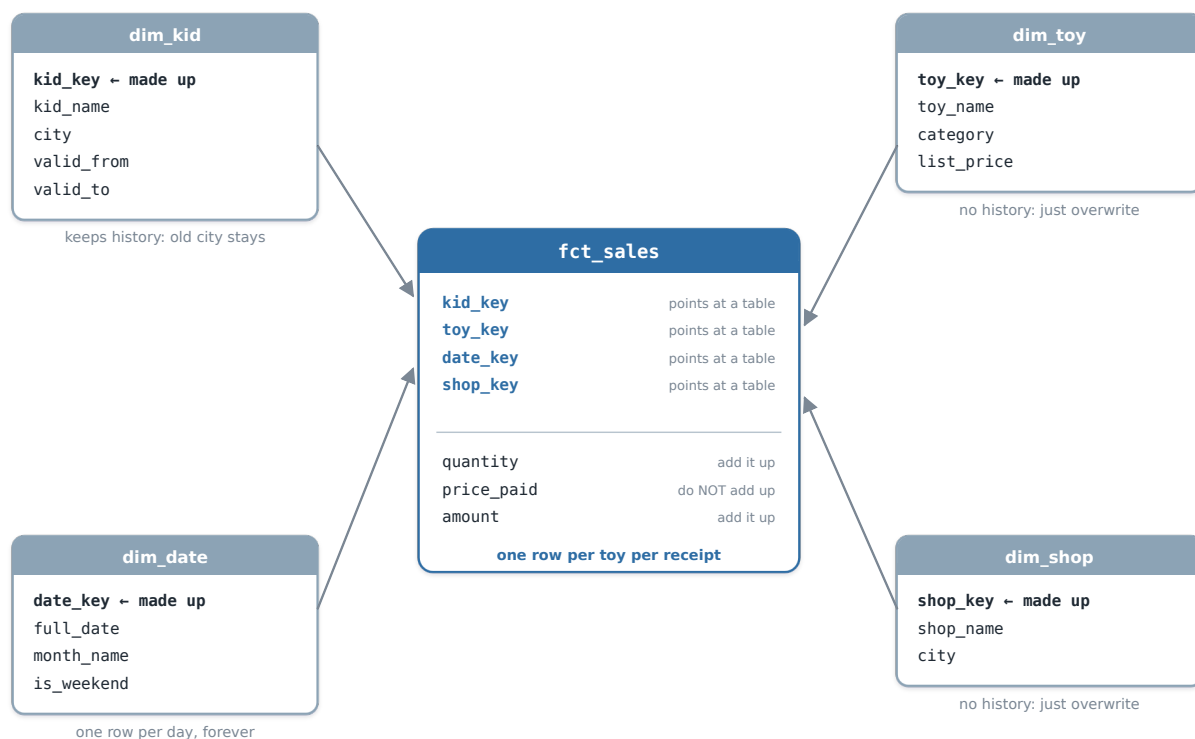
Run the notebook twice. `silver_sales` should still say 6 — it overwrites. `silver_kids_history` will grow each run, which is correct: it is a diary, not a snapshot.

5 Draw the star before you build it

Now you decide the shape. Start from the questions people will ask:

- How much money did we take, by toy type, by month?
- Which shop sold the most last week?
- How many children came back and bought again?

Look at the words. **How much money** and **how many** are things you add up. **By toy type, by shop, by month** are things you slice by. That is your whole design: things you add up go in the middle, things you slice by go around the outside.



Four small tables around one big one. That shape is why it is called a star.

One big table in the middle. Four small ones around it.

Word	What it means, simply
Fact table	The big middle one. One row per thing that happened.
Dimension	A small outside one. The things you slice by.
Key	A made-up number that joins a small table to the big one.
Add it up	Safe to total. Quantity, money.
Do NOT add up	Price is one of these. Adding up prices gives a meaningless number.

Two choices worth understanding

Why `dim_kid` keeps history and `dim_shop` does not. If Ben moves from Toronto to Hamilton, last year's sales should still say Toronto — that is where he lived when he bought them. So `dim_kid` keeps both, with dates saying when each one was true. But if a shop's name was just spelled wrong and got fixed, nobody wants to see the typo again. Overwrite it and move on.

Why the price is in the big table. Toy prices change. If the big table only points at the toy, and you look the price up later, then every old sale quietly changes price whenever the shop changes its prices — and last month's takings move. Store what was actually paid, on the row, at the time. Then it can never move.

6 Connect dbt to the Warehouse

```
pip install dbt-core dbt-fabric
dbt --version          # you should see "fabric" in the list
```

Then write this file at `~/.dbt/profiles.yml`:

```
retail_fabric:
  target: dev
  outputs:
    dev:
      type: fabric
      server: "xxxx.datawarehouse.fabric.microsoft.com"
      database: "wh_retail"          # the WAREHOUSE. never the lakehouse.
      schema: "dbo"
      authentication: "CLI"         # run `az login` first
      threads: 4
```

Then tell dbt where Silver lives. This is the bit that joins the two halves together:

```
# models/staging/sources.yml
version: 2
sources:
  - name: lakehouse
    database: lh_retail             # ← reaches across into the Lakehouse
    schema: dbo
    tables:
      - name: silver_sales
```

YOU SHOULD SEE

dbt debug saying **All checks passed.**

Keep `threads` at 4. Higher numbers make Fabric runs hang for minutes at "listing schemas" — it looks broken, but it is just queuing.

7 Build the four small tables

One file each. Give every row a made-up number as its key:

```
-- models/marts/dim_toy.sql
SELECT
  ROW_NUMBER() OVER (ORDER BY toy_name) AS toy_key,
  toy_name,
  category,
  list_price
FROM {{ ref('stg_toys') }}
```

`dim_shop` and `dim_date` look the same. `dim_kid` is the one that keeps history, and dbt has a tool for that called a snapshot:

```
{% snapshot kid_snapshot %}
{{ config(target_schema='snapshots', unique_key='kid_name',
  strategy='check', check_cols=['city']) }}
SELECT kid_name, city FROM {{ ref('stg_kids') }}
{% endsnapshot %}
```

NEVER RUN `--FULL-REFRESH` ON A SNAPSHOT

It deletes all the history and starts again. There is no undo. This is the most damaging command in dbt, and people usually type it by accident while refreshing everything else.

8 Build the big table

Its only job is to swap real names for the made-up key numbers, and work out the money.

```
-- models/marts/fct_sales.sql
SELECT
    d.date_key,
    COALESCE(k.kid_key, -1) AS kid_key,
    COALESCE(t.toy_key, -1) AS toy_key,
    COALESCE(s.shop_key, -1) AS shop_key,

    f.receipt_no,
    f.quantity,
    f.toy_price,
    f.quantity * f.toy_price AS amount

FROM {{ ref('stg_sales') }} f
LEFT JOIN {{ ref('dim_date') }} d ON d.full_date = f.sale_date
LEFT JOIN {{ ref('dim_toy') }} t ON t.toy_name = f.toy_name
LEFT JOIN {{ ref('dim_shop') }} s ON s.shop_name = f.shop_name
LEFT JOIN {{ ref('dim_kid') }} k
    ON k.kid_name = f.kid_name
    AND f.sold_at >= k.valid_from      -- pick the version of Ben
    AND f.sold_at < k.valid_to        -- that was true back then
```

The bit

What goes wrong without it

`LEFT JOIN`, not plain
`JOIN`

A plain join silently deletes any sale whose toy is missing. Your money total drops and nothing warns you.

`COALESCE(..., -1)`

Sends the odd ones to a row called "Unknown" so you can still count them, instead of losing them.

The two date lines on
`dim_kid`

Without both, Ben having lived in two cities makes every one of his sales count twice. Your money doubles and still looks believable.

9 Check it is right

Two checks catch most real problems. Both pass when they find nothing.

```
-- tests/check_no_duplicates.sql
-- our rule was: one row per toy per receipt
SELECT receipt_no, toy_name, COUNT(*)
FROM {{ ref('fct_sales') }}
GROUP BY receipt_no, toy_name
HAVING COUNT(*) > 1
```

```
-- tests/check_money_matches.sql
-- the Gold total must equal the Silver total
SELECT g.total, s.total
FROM (SELECT SUM(amount) AS total FROM {{ ref('fct_sales') }}) g,
      (SELECT SUM(quantity * toy_price) AS total FROM {{ ref('stg_sales') }}) s
WHERE ABS(g.total - s.total) > 0.01
```

IF YOU ONLY EVER WRITE ONE TEST, WRITE THE MONEY ONE

A total that quietly disagrees with where it came from is the bug that lives longest, because the number still looks fine. Nobody spots a 3% drop. This test spots it the same night.

Then run everything with `dbt build` — not `dbt run`. `build` tests each table as it goes and stops at the first problem, so a broken table never gets built on top of.

YOU SHOULD SEE

Every model **PASS**, and a money total of 194.46 — the same number you got in Guide 05 on your laptop. Same data, same answer, different machine.

10 Run it nightly and put a chart on it

Make a pipeline in Data Factory with three steps in a row:

```
Copy files → Run notebook → Run dbt build
```

Then in the Warehouse choose **New semantic model**, add your five tables, and draw a line from each small table to the big one. Every line must be **one-to-many and one direction only**, from small to big.

TWO SETTINGS THAT MATTER MORE THAN THEY LOOK

Make the pipeline fail when a test fails. A pipeline that says "success" while tests are failing is worse than no pipeline — it makes people trust bad numbers.

Never use two-way relationships in the report. They look handy and they produce wrong numbers that nobody can explain.

5. Finished checklist

Layer	Check
Bronze	Every source table has a primary key, so nothing is skipped
	All tables show Running , and row counts match the source
	You have not tried to change anything in the mirror
Silver	Anything that changes is written down with a date on it
	Running it twice gives the same numbers for sales
	You could answer "what did this look like last month?"
Gold	Your grain sentence is written in the model description
	Every small table has a made-up key, and the big table joins only on those
	The history join has both date lines
	Missing things go to <code>-1</code> , not blank
Tests	Duplicate check on the big table
	Money check, Gold against Silver
Running it	Pipeline fails when a test fails
	Snapshots left out of any full refresh

6. When it goes wrong

What you see	What it really means
dbt says it cannot write	You pointed <code>database:</code> at the Lakehouse. It must be the Warehouse.
<code>Invalid object name lh_retail.dbo...</code>	The Lakehouse was not added to the Warehouse, or they are in different workspaces.
dbt hangs at "listing schemas"	Too many threads. Set <code>threads: 4</code> .
Login works but every model fails	You are not a workspace admin. dbt needs that today.
Money in Gold is double Silver	The history join is missing one of its two date lines.
Money in Gold is less than Silver	A plain <code>JOIN</code> threw rows away. Use <code>LEFT JOIN</code> .
A table never appeared in the mirror	It has no primary key at the source. Mirroring cannot follow it.
Spark cannot read the shortcut	You need Read all OneLake data on the mirrored database. Admin alone is not always enough.
A row vanished from Silver	It was deleted at the source, so the mirror dropped it. This is mirroring working correctly — and why Silver keeps its own dated copy.
Yesterday's address is gone	You overwrote instead of appending. The mirror cannot give it back. Nothing can.
History disappeared	Someone ran <code>--full-refresh</code> on the snapshot. It cannot be recovered.

7. Where each idea came from

In this project	Explained in
Bronze, Silver, Gold	Guide 05, concepts 1-4
Cleaning and conforming in Silver	Guide 05, concept 3
Keeping history the source throws away	Guide 05, concept 6
Running it twice safely	Guide 05, concept 5
The money check	Guide 05, concept 7
One row per toy per receipt	Guide 01, concept 2
Made-up keys	Guide 01, concept 4
The star shape	Guide 01, concept 6
Keeping history	Guide 01, concept 9
Money doubling	Guide 01, concept 10
Snapshots and tests in dbt	Guide 06, concepts 7 and 9